
Does the Source of Carrier Image Affect Steganographic Detectability?

Final Presentation

Abdul Moiz Akbar | Malo Coquin | Daria Gjonbalaj | Nico Muller-Spath
Jimena Narvaez del Cid | David Wicker | Nikolas Zouros

Department of Advanced Computing Sciences
Maastricht University

Project 2.2 | May 2026

Agenda

1. Motivation recap
2. Research questions
3. Experimental design and methods
4. Statistical analysis
5. Results across RQ1–RQ5
6. Limitations and contribution

Motivation Recap

- Steganalysis detectability depends on carrier-image statistics, not only on embedding logic (Petitcolas et al., 1999; Fridrich & Kodovsky, 2012).
- Photographic carriers dominate benchmarks, yet diffusion outputs are now everywhere and carry distinct statistical traces (Corvi et al., 2023).
- Closest prior work studies AI carriers under adaptive embedding and reports P_E , not AUC (Méreur et al., 2024).

Core question

Do detectors validated on photographs remain equally effective when the carrier is ML-generated, under identical embedding settings?

Research Questions

RQ1 (Primary, confirmatory)

Does carrier source (real vs. pooled ML) affect detectability under matched embedding settings?

RQ2 (Primary, confirmatory)

Within ML carriers, does the choice of generator (SDXL vs. FLUX) change detectability?

RQ3 (Exploratory)

Does payload size change the detectability gap between real and ML carriers?

RQ4 (Exploratory)

Do the spatial (LSB+PNG) and frequency (DCT-LSB+JPEG) branches show different carrier-source effects?

RQ5 (Verification)

Does AES-256-CBC payload encryption affect detectability? Expected null result.

Datasets and Factorial Design

Datasets (1,500 images)

- Real: 300 COCO + 200 Flickr30k
- ML-A: SDXL 1.0 (500 caption-matched)
- ML-B: FLUX.1-schnell (500 caption-matched)
- Grayscale 512×512, dual storage: PNG and JPEG Q=95

$3 \times 2 \times 3 \times 2$ factorial

- Source: real / ml_a / ml_b
- Method: LSB / DCT-LSB
- Payload: low (0.25) / med (0.50) / high (0.75)
- Encryption: plain / AES-256-CBC

Caption-matched, not pixel-matched

ML generators receive the same captions as real images; subject matter held constant, carrier statistics vary. Per group: 3 covers $\times$ 12 stego conditions $\Rightarrow$ 36 stego images, $\sim$ 60 detector evaluations.

Embedding Methods

Spatial: LSB substitution

- Replace pixel LSB, $k = 1$, row-major
- Lossless PNG storage required
- Detected by RS, χ^2 , Sample Pairs

Frequency: DCT-LSB (JSteg-style)

- Flip LSB of *non-zero* quantised AC coefficients
- Skip DC and $|c| = 1$ (Westfeld-pair stable)
- JPEG Q=95, single quantisation pass

Why classical, not adaptive (HILL / UNIWARD / MiPOD)

Adaptive schemes adapt to image content, conflating embedding evasion with carrier statistics. Classical fixed-position embedding isolates the carrier-source signal we want to measure.

Detector Choices and Mechanisms

Spatial branch

- **RS Analysis** (Fridrich 2001): regular/singular pixel-group counts
- χ^2 **attack** (Westfeld 1999): $\{2k, 2k+1\}$ pairs-of-values
- **Sample Pairs** (Dumitrescu 2003): trace multisets on adjacent pairs

Frequency branch

- χ^2 **DCT** (Westfeld 1999): pairs on quantised AC coefficients
- **Calibration** χ^2 (Fridrich 2003): non-block-aligned crop + recompress reference

Why training-free

Classical statistical detectors have no learned parameters. AUC differences between carrier groups reflect carrier statistics, not a training-set mismatch – critical for an unbiased real-vs-ML comparison.

Pipeline Structure

Five sequential stages

- Data: real downloads + ML generation
- Manifests: payloads + stego (seeded)
- Embedding: stego images + PSNR/SSIM/FSIM
- Detection: 5 classical detectors on cover+stego
- Aggregation: AUC, contrasts, verdicts, power

Run isolation

- Each run in its own `runs/<id>/`
- Config frozen in `config.json`
- Manifests record every condition + seed
- Re-runs idempotent: done work is skipped

`python run.py <profile>` produces every cover, stego, CSV, figure, and verdict in a self-contained directory; no manual post-processing.

Web Platform and Live Monitoring

Architecture

- Local Python HTTP server (stdlib only)
- Single-page browser client, no build step
- Endpoints: list / launch / preview / kill / detail
- Launch spawns the runner as a subprocess

Live observability

- Subprocess stdout via Server-Sent Events
- Browser terminal panel shows progress live
- Preview validates config before any compute
- Run-detail surfaces metrics, figures, gallery

Power estimate in the drawer: Hanley–McNeil computed live in JavaScript, colour-coded against the proposal threshold $\delta = 0.05$.

Statistical Analysis

Tests

- DeLong paired ROC test for correlated AUCs (DeLong 1988)
- Bonferroni–Holm correction across the confirmatory family
- 95% CIs for exploratory comparisons (RQ3, RQ4)
- Equivalence margin ± 0.025 AUC for RQ5

Power (Hanley–McNeil)

- $\alpha = 0.05$, target power 0.80
- Confirmatory family size: 15 strata per RQ
- Min detectable δ_{AUC} at $N=500$: RQ1 ≈ 0.057 , RQ2 ≈ 0.066 , RQ5 ≈ 0.050

Pragmatic boundary

$N=500$ honours the proposal threshold $\delta_{\min}=0.05$ for the primary (RQ1) and verification (RQ5) questions. RQ2 is a documented power limitation.

RQ1 – Real vs. Pooled ML

Test

Paired DeLong per stratum (detector $\times$ method $\times$ payload), Holm-adjusted across 15 strata.

Headline finding

- Pooled Δ_{AUC} : [*to fill from exp1_*_contrasts.csv*]
- Significant strata (Holm $p < 0.05$): [$n / 15$]
- Verdict: [*supported / mixed / not supported*]

Interpretation

- Direction of the effect, consistent across detectors or not.
- Where is the gap strongest – spatial or frequency branch?
- Cite exp1_real_vs_pooled_ml.png forest.

RQ2 – SDXL vs. FLUX within ML

Test

Paired DeLong, same 15-stratum family, Holm-adjusted. Caption-matched cover pairs across the two generators.

Headline finding

- Pooled Δ_{AUC} : [*from* exp2*_contrasts.csv]
- Significant strata: [*n / 15*]
- Verdict: [*supported / mixed / not supported*]

Power caveat

At $N=500$ the minimum detectable δ for RQ2 is ≈ 0.066 . Smaller generator-specific effects are below the resolution of this study.

RQ3 and RQ4 – Exploratory

RQ3: Payload interaction

- Real-vs-ML gap by payload level (low / med / high).
- Trend: [*increasing* / *non-monotonic* / *flat*]
- Cite `exp3a_payload_level_auc.png` and the `source`×`payload` heatmap.

RQ4: Branch interaction

- Spatial source gap minus frequency source gap, per detector and payload.
- Pooled $\Delta\Delta$: [*from* `exp4_*_contrasts.csv`]
- Cite `exp4_spatial_vs_frequency.png`.

Reporting convention

RQ3 and RQ4 are exploratory: we report point estimates and 95% CIs, no Holm correction, no significance verdict.

RQ5 – Encryption Invariance (verification)

Test

Paired DeLong between plain and AES-256-CBC stego on shared cover groups. Equivalence margin ± 0.025 AUC per stratum.

Headline finding

- Strata within margin: $[n / 45]$
- Pooled Δ_{AUC} : [*near zero, expected*]
- Verdict: encryption invariant / not supported

Why the null matters

A non-null result would indicate the detectors are reacting to payload structure rather than the embedding distortion itself. The null is the proposal-aligned outcome.

Limitations

Acknowledged deviations

- ml_b: PixArt- α replaced with FLUX.1-schnell (PixArt unavailable on HF Inference API).
- Power: RQ2 effective $\delta_{\min} \approx 0.066$ at $N=500$, not 0.050.

Scope-bounded

- Two generator families only – cannot disentangle architecture from training data in RQ2.
- Grayscale only; no colour information.
- Two DCT-branch detectors vs. three spatial.

Contribution and Minimum Deliverable

What this study contributes

- Caption-matched dataset across three carrier sources under identical embedding.
- Training-free AUC-based comparison with DeLong and Holm correction.
- Pre-computed RQ verdicts and pilot-based power analysis as reproducible artefacts.

Minimum deliverable met

- One encryption scheme (AES-256-CBC). ✓
- Two embedding approaches: spatial LSB and frequency DCT-LSB. ✓
- Defensible answers to RQ1 and RQ2. ✓

Thank you

Questions and Discussion

Code and run artefacts: `runs/<id>/`
